

Lowerbeam Coding Architecture

Executive recommendation

Build Lowerbeam Coding as a **project-scoped, headless agent runtime**, controlled by Lowerbeam and presented through a new Coding tab. Preserve the existing local model-management experience. Keep the agent loop out of React, place every action behind a trusted permission broker, and run project commands within an operating-system sandbox. A model should propose actions; Lowerbeam should decide what those actions are allowed to reach.

The preferred first engine candidate is Pi's lightweight agent core, behind a Lowerbeam-owned adapter. Use it for the model/tool iteration loop rather than adopting its complete coding application. Compare that candidate against an OpenCode headless server using the same local models and tasks before committing. OpenCode is the stronger shortcut when speed of integration outweighs control over the execution model. Neither integration has been tested in Lowerbeam, so this is an architectural recommendation, not a performance ranking.

The central investment should be the product boundary: workspace selection, enforceable capabilities, durable task state, accurate diffs, model scheduling, cancellation, and evidence of verification. Those components remain valuable when the model or agent engine changes. A large prompt, a tool protocol, or a collection of specialist agents cannot substitute for them.

Scope and confidence

This assessment uses public source and documentation inspected on 8 September 2026. Lowerbeam's inspected main branch was headed by commit [2b6c51d5ed1456cd4aa91d07adf528b31775c0e6](#); its package file reported version 0.4.1. Inspection was static and selective: no checkout was built, no tests were executed, and no model or integration benchmarks were run. The findings are not a security audit.^{1,2}

The working assumptions are local inference by default, a selected software project rather than unrestricted desktop automation, and an initial single-user desktop experience. Linux is the proposed first execution target because the current packaging configuration specifies Linux artifacts; it is not an assumption about any particular user's computer. Other operating systems require their own validated execution backend.³

Confidence is high in separating orchestration, permissions, and execution from the interface. Confidence is moderate in the proposed engine choice. Actual task success, latency, memory use, and compatibility remain unknown until evaluated on the intended models and machines.

1. Prompt, context, and harness engineering

These are complementary layers rather than competing approaches. Prompt engineering shapes instructions. Context engineering decides which information the model receives at each step. Harness engineering builds the software and feedback mechanisms that make a sequence of model decisions useful and controllable. Anthropic's context guidance explicitly distinguishes prompt writing from managing the broader information supplied to an agent; OpenAI's harness account also includes repository legibility, documentation, and mechanical checks.^{4,5}

¹ Lowerbeam, [Repository commit 2b6c51d](#), 8 September 2026.

² Lowerbeam, [package.json](#), inspected revision.

³ Lowerbeam, [electron-builder.yml](#), inspected revision.

⁴ Anthropic, [Effective context engineering for AI agents](#), 29 September 2025.

⁵ OpenAI, Ryan Lopopolo, [Harness engineering](#), 11 February 2026.

Layer	Lowerbeam example	Boundary
Prompt	Read relevant code before editing; preserve unrelated changes; report tests honestly.	Guides behaviour, but cannot grant or enforce authority.
Context	Supply the task, selected files, repository conventions, relevant test output, and current progress.	Determines what the model can reason about, not what the process can access.
Harness	Validate tool calls, enforce workspace scope, execute commands, save state, show diffs, and stop work.	Owns the actual operating rules and evidence.

For this feature, the highest-risk work is in the harness. The smallest useful end-to-end loop is: inspect the project, choose a bounded action, authorise it, execute it, observe the result, and either continue or stop. A successful answer is not merely an explanation of a fix. It is a reviewable change accompanied by a record of what was checked and what remains uncertain.

2. Lowerbeam's existing foundation

Lowerbeam already combines local llama.cpp lifecycle management with model configuration, streamed conversations, and tool support. Its TypeScript/Electron architecture is a reasonable host for a separate local runtime. There is no reason to replace the desktop application simply to gain a coding agent.⁶

The important extension points are more specific than the new tab itself:

Existing area	Observed behaviour	Architectural consequence
chatStore.ts	The renderer owns the current completion/tool loop, with a four-round tool limit and folding of older tool results.	Keep normal chat working; introduce a separate coding job lifecycle rather than expanding this store into an execution service.
chatClient.ts	Streams OpenAI-style chat completions and assembles tool-call fragments; sends llama.cpp-specific sampling options.	Extract a reusable model transport/adaptor without importing renderer state into the worker.
ipc.ts	Validates requests and dispatches built-in or MCP tool calls.	Add run identity, workspace grants, sender checks, and per-action policy before exposing local file and execution tools.
mcp.ts	Starts configured servers as child processes and passes an environment derived from the host process.	MCP connectivity is already useful, but should not become the default unrestricted execution route for Coding.

These observations come from the corresponding source files. They identify design boundaries, not demonstrated exploits or tested failures.^{7,8,9,10}

The main window already enables context isolation and renderer sandboxing with Node integration disabled. Preserve those protections. They do not establish an operating-system boundary around future shell commands. The IPC dispatch wrapper inspected does not itself validate the sender; the Coding API should explicitly bind requests to the authorised application frame and validate both identity and payload.^{11,12}

⁶ Lowerbeam, [README.md](#), inspected revision.

⁷ Lowerbeam, [chatStore.ts](#), inspected revision.

⁸ Lowerbeam, [chatClient.ts](#), inspected revision.

⁹ Lowerbeam, [ipc.ts](#), inspected revision.

¹⁰ Lowerbeam, [mcp.ts](#), inspected revision.

¹¹ Lowerbeam, [index.ts: main process](#), inspected revision.

¹² Electron, [Security: recommended practices](#), undated.

The current separation is suitable for conversation plus selected tools. Coding introduces a different lifetime: a task may edit files, wait for permission, spawn grandchildren, survive a renderer refresh, and encounter a crash between two changes. Those responsibilities belong in a backend job system.

3. Engine selection and build-versus-integrate

Preferred candidate: Pi core with Lowerbeam-owned tools

Pi's current agent core exposes stateful tool execution, events, context transformation, cancellation facilities, and configurable execution ordering. Its SQLite session backend is separate rather than mandatory. These are useful primitives for embedding an agent without adopting an existing coding interface.¹³

The recommendation is to use the low-level core and model adapter, not to enable the complete CLI's default tools or configuration discovery. Pi's higher-level coding SDK includes built-in file and shell tools as well as session management; that is convenient, but it also means deciding which execution authority and configuration behaviour to inherit.¹⁴

Lowerbeam should supply the tools, application state, approved instructions, and storage locations. Pin the dependency version, expose a narrow engine interface, and initially select sequential tool execution. This keeps write ordering and cancellation easier to reason about. Parallel independent reads can be evaluated later.

Pi documents custom OpenAI-compatible model endpoints, which makes a llama.cpp integration plausible. Compatibility documentation does not establish that every GGUF model can complete reliable multi-step tool work. A focused integration trial must verify streaming, tool-result formatting, sampling controls, abort propagation, and context limits.¹⁵

Alternative: OpenCode as a headless sidecar

OpenCode exposes a headless HTTP server and documents a llama.cpp provider. It therefore offers a credible way to connect Lowerbeam's interface to a more complete existing agent rather than build the runtime experience from primitives.^{16,17}

The tradeoff is that another application owns more of the tool, session, configuration, and lifecycle behaviour. Its documented permission defaults are permissive for most actions, so a Lowerbeam integration must supply explicit policy rather than inherit defaults.¹⁸

For this route, bind the service to loopback, enable authentication with an ephemeral credential, isolate its configuration, and run it under an appropriate workspace sandbox. Do not assume Lowerbeam's own tool broker can intercept a separate engine's internally executed commands. Either route execution through supported extension points or contain the whole sidecar with an independently enforced boundary.

Decision matrix

Option	Best fit	Principal cost
Pi core plus owned runtime	A native Lowerbeam experience with precise control over local models, tools, and permissions.	More product infrastructure must be built and maintained.

¹³ Pi project, [Agent core README](#), undated.

¹⁴ Pi project, [Coding agent SDK documentation](#), undated.

¹⁵ Pi project, [Custom model configuration](#), undated.

¹⁶ OpenCode, [Server documentation](#), undated.

¹⁷ OpenCode, [Providers: llama.cpp](#), undated.

¹⁸ OpenCode, [Permissions](#), undated.

5. Permissions and execution containment

A capability policy rather than an instruction

A useful first boundary is one explicitly selected project, one isolated task workspace, and one set of approved capabilities. Every tool call should be validated against both its schema and that run's current policy. Unknown tool names must fail closed. Neither the model nor a repository instruction file may expand its own access.

Offer a small number of understandable modes: inspect without arbitrary command execution; edit an isolated copy; run commands inside a sandbox; and request narrowly scoped additional access. Installations, network use, access outside the project, destructive Git operations, and publishing should not silently accompany an ordinary code change.

The rationale is environmental containment, not confidence that instructions will always be followed. Anthropic's published containment account distinguishes supervision from restricting what an agent can reach, and describes failures involving pre-trust project configuration and permitted-domain exfiltration. The lesson for Lowerbeam is to establish authority before loading executable project configuration and to treat outbound access as a capability grant, not merely a domain string.²¹

Tool and process rules

Start with a compact tool surface: discover/search files, read bounded ranges, propose/apply patches, inspect changes, and start/wait/cancel approved commands. Keep complete output as local artifacts while returning bounded excerpts to the model. Use strict tool schemas and execute only completed, validated tool calls, never code guessed from a streamed Markdown fence.

Bind filesystem operations to canonical granted paths. Handle symlinks, traversal, non-existent destination parents, and race conditions. A prefix comparison alone is not a complete filesystem boundary. Keep credential directories, browser profiles, Lowerbeam's own state, sensitive environment files, and Git control files outside the normal grant. A sandbox should supply an additional operating-system enforcement layer when a process bypasses the intended tool API.

For subprocesses, provide an explicit working directory, a restricted environment, time and output limits, and a way to terminate the full process tree. Avoid inherited cloud credentials and privileged sockets. A command named `npm test` is still repository-controlled code; a reassuring command prefix is not proof of safe behaviour. Render terminal output defensively rather than treating it as trusted interface markup.

Sandbox implementation

Anthropic's sandbox-runtime is a relevant reusable candidate. Its current repository documents Linux, macOS, and Windows implementations and separate filesystem/network controls. Its broad default read access is especially important: merely adopting the library would not implement a project-only confidentiality policy. Configure and test the read boundary as well as writes and egress.²²

Start with the platform Lowerbeam actually packages and support a capability probe. Missing dependencies or an unavailable isolation mechanism should leave Coding in read-only mode or fail the execution request, not silently run unsandboxed. Validate each operating system separately. A VM may be appropriate for stronger isolation later, but comes with additional provisioning, resource, and observability work.

²¹ Anthropic, Max McGuinness et al., [How we contain Claude across products](#), 25 May 2026.

²² Anthropic, [sandbox-runtime repository and README](#), undated.

Do not provide an ordinary approval button that turns a scoped job into unrestricted host execution. Additional access should create an explicit, visible change in the run's grant and should remain unavailable where policy forbids it. Approval is a user-experience mechanism; containment is the enforceable boundary.

MCP is an extension boundary

MCP should remain optional and selectively enabled per project or run. The current specification's security guidance covers local server execution and the need for appropriate consent and containment. A tool transport does not itself certify a server's behaviour or constrain its host privileges.²³

Treat a local MCP server as installed software. Avoid starting every configured server for every coding task, inheriting the full environment, or exposing all discovered tools to the model. Tool responses, repository files, test logs, and web results are data, even when delivered by an approved connector.

6. Workspaces, changes, and recovery

Protect the original project

Prefer an isolated task workspace with a captured baseline. For Git repositories, a worktree can be useful; Git documents that linked worktrees have separate working directories while sharing repository data. It is therefore not an operating-system sandbox and not complete repository independence.²⁴

A worktree created at HEAD does not automatically contain the user's uncommitted edits. Make the starting policy explicit: require a clean baseline, or capture the selected tracked and untracked changes into the task workspace. Never silently discard dirty state. Keep one writer per workspace initially.

Each edit should have preconditions, such as an expected content hash or an exact matched range. Reject stale patches and reread rather than applying them to a file that has changed underneath the agent. Stage and review multi-file changes; do not claim that a series of ordinary filesystem writes is one atomic transaction.

The Changes view should show created, modified, and deleted files plus the baseline used. Applying the result back to the original project must detect conflicts. Undo should target the agent's recorded changes and check for later user edits. Never implement ordinary undo as an indiscriminate hard reset. Local file rollback cannot reverse a package's network activity or writes to an external database.

Durable jobs rather than saved transcripts

Use an append-only event journal and atomic checkpoints for the first implementation. JSONL is a reasonable proposed starting point; evaluate SQLite if querying or concurrency justifies the additional storage integration. Store journals, policies, and authoritative checkpoints outside the agent-writable repository.

Persist critical transitions before side effects and record their outcomes afterwards. Coalesce token deltas for interface responsiveness rather than synchronously writing every token. On restart, reconcile the journal with the workspace. A crash after a command started but before completion was recorded means its result is uncertain; it does not justify automatically running the command again.

Cancellation must stop model inference, queued actions, pending approvals, and all spawned command descendants. Test each path. A renderer refresh should reconnect to the existing job. Application quit should follow an explicit shutdown/recovery policy; continuing after the application closes would require a separate service design and is outside this first proposal.

²³ Model Context Protocol, [Security best practices](#), 2026-07-28 documentation.

²⁴ Git, [git-worktree documentation](#), undated.

7. Context and local-model integration

Build a bounded working set

Keep a small stable policy prefix, the user's actual task and acceptance conditions, a compact repository map, relevant project conventions, selected file excerpts, recent tool results, and a structured current-state summary. The recommendation is progressive retrieval: begin with names and text search, then inspect only the files needed to answer the next question. Do not begin by embedding the entire repository or inserting all its contents into every request.

Instructions such as an approved AGENTS.md can describe build commands and conventions, but cannot override application policy. Keep factual notes distinct from instructions. Add a symbol index or language server only when evaluations demonstrate that search and targeted reads are missing important relationships.

Reserve context for the next tool result and the next model response. Compact before exhaustion, at a safe turn boundary after outstanding tool results have been attached to their calls. Preserve the goal, constraints, changed files, verification status, unresolved errors, and next action. Summaries are fallible: reread relevant current files before consequential edits. Keep full logs outside the prompt and use references to retrieve them when necessary.

Test the model-server-template combination

llama.cpp documents both native and generic function-call handling and the importance of a tool-aware chat template; it also notes potential performance degradation from extreme KV-cache quantisation. This means endpoint compatibility alone is an insufficient readiness test.²⁵

Create a capability record for the exact model file/hash, quantisation, chat template, llama.cpp build, context configuration, sampling settings, and tool schema version. A model that writes convincing code may still emit malformed arguments, repeat an action after failure, mishandle tool results, or fail to stop. Distinguish tested Coding-ready configurations from experimental or chat-only configurations.

Preserve Lowerbeam's inference controls through the new adapter, including non-standard sampling fields where supported. Do not silently switch to a cloud model after a local failure. Treat any remote provider as a separate explicit data-sharing choice.

Add a model lease: a coding run pins the model configuration it depends on. Other tabs should queue or request an orderly transition instead of stopping or replacing that model midway through a task. Start with one active reasoning job; introduce additional concurrency only after measuring resource contention and effective context per slot.

Avoid a multi-agent default

Begin with one agent using a repeatable inspect-edit-test loop and deterministic verification. Specialist planners, reviewers, and evaluators can be useful for some tasks, but their value must exceed their additional inference, state, and coordination costs on the target machine. Anthropic's long-running harness experiments also show that useful scaffolding depends on the model and should be reevaluated as capabilities change.²⁶

The proposed rule is to add another agent only to address a measured failure category. Do not create a permanent planning/review bureaucracy to compensate for missing file tools, poor context selection, or weak tests.

²⁵ llama.cpp, [Function calling documentation](#), undated.

²⁶ Anthropic, Prithvi Rajasekaran, [Harness design for long-running application development](#), 24 March 2026.

8. Interface, interoperability, and delivery sequence

The Coding tab should communicate what the system is doing without requiring the user to read every tool argument. Its essential views are the task conversation, current status, permission requests, changed files, command output, and verification summary. Show the selected workspace, model, and access mode throughout the run. The main actions are stop, respond to a scoped request, inspect changes, and apply or discard the result.

Use a small internal protocol initially. ACP is relevant to future interoperability between a coding client and an external agent; MCP addresses a different connection, between an agent/client and tools. Pin supported protocol versions and negotiate capabilities when adding adapters rather than making either protocol responsible for Lowerbeam's security model.²⁷

Delivery stages

Stage 0 - Compatibility and containment trials. Define the engine contract and threat model. Exercise Pi and OpenCode against the same small task set and local model configurations. Verify the sandbox backend on the intended Linux packaging path. Select the engine using observed compatibility and integration effort, not reputation.

Stage 1 - Read-only project intelligence. Add project selection, the backend job lifecycle, bounded search/read tools, policy decisions, and reconnectable events. Verify that the model can locate relevant code and explain it. Opening an untrusted project must not execute its hooks or configuration.

Stage 2 - Reviewable edits. Add isolated workspaces, baseline capture, preconditioned patches, the Changes view, conflict handling, and safe discard. Keep command execution disabled until its containment and process lifecycle are ready. This produces a useful feature before granting arbitrary repository code execution.

Stage 3 - Sandboxed verification and recovery. Add command execution, cancellation, test evidence, installation/network approval paths, interruption recovery, and controlled apply-back. This is the first stage that should be presented as an autonomous edit-and-test workflow.

Stage 4 - Evidence-led expansion. Add selected MCP servers, language servers, additional platforms, optional cloud engines, or external-agent interoperability only after the core release gates pass. Schedule each addition as a new capability with its own threat and recovery tests.

9. Evaluation and release gates

A proposed initial suite of roughly 20-30 tasks is enough to expose recurring problems, but not to establish universal safety or quality. Test the final workspace state and recorded actions, not just whether the final message sounds correct. Anthropic's agent-evaluation guidance similarly separates tasks, trajectories, and outcomes and describes executable checks for coding agents.²⁸

Evaluation family	Representative checks	Proposed release requirement
Task quality	Locate relevant code; fix a small bug; make a cross-file change; recover from a failed test.	Changes meet explicit acceptance checks without unrelated regressions.
Authority	Poisoned README or test output; secret canary outside scope; symlink escape; denied network.	No unauthorised access in the tested cases; denials remain effective after retries.
File integrity	Dirty starting tree; concurrent user edit; stale patch; partial multi-file change.	Preserve the baseline, reject conflicts, and expose uncertainty rather than overwrite.

²⁷ Agent Client Protocol, [Protocol overview](#), version 1 documentation.

²⁸ Anthropic, [Demystifying evals for AI agents](#), 9 January 2026.

Evaluation family	Representative checks	Proposed release requirement
Lifecycle	Cancel a child process tree; restart during execution; model disconnect; renderer reload.	No orphaned tested process, silent replay, or falsely completed job.
Model compatibility	Fragmented tool calls; invalid arguments; tool failure; context compaction; repeated action.	Validated recovery or an explicit stop instead of uncontrolled execution.

For every run, record the configuration identity, starting state, commands, exit codes, selected output, changes, and final verification status. Separate pre-existing test failures from newly introduced failures. Treat modified or deleted tests as reviewable changes rather than automatic evidence of improvement; keep acceptance tests outside the agent's writable area where possible.

Measure completed-task success, unwanted changes, approval frequency, wall-clock time, peak resource use, context growth, and cancellation/recovery behaviour. Tokens per second alone is not a coding-product metric. Compare engines using the same workload, models, permissions, and resource budgets. Report variation across repeated runs rather than presenting one successful demonstration as a benchmark.

The first release should satisfy a concrete scenario: select a project, request a bounded fix, inspect the plan and diff, see recorded checks, and apply the change without losing unrelated work. A malicious instruction in project content must not grant new access. A stop request must terminate execution. An interrupted task must return with an honest, reconcilable status.

Final decision

Proceed with a Lowerbeam-owned runtime boundary and a Pi-core integration trial, using OpenCode as the explicit alternative. Keep the architecture single-agent, local-first, project-scoped, and test-driven. Adopt mature sandbox primitives rather than inventing containment, but own and validate the policy around them.

The engine may be replaceable; the trust and recovery contract must not be. Lowerbeam's distinctive value would be a local coding environment that makes its access, changes, resource use, and evidence understandable, rather than merely making a local model behave like an unattended terminal user.

Sources

All online sources below were inspected on 8 September 2026. Repository links identify the inspected Lowerbeam commit where applicable. Undated technical documentation describes the version available at inspection and may change.

1. Lowerbeam. [Repository commit 2b6c51d](#). 8 September 2026.
2. Lowerbeam. [package.json](#). inspected revision.
3. Lowerbeam. [electron-builder.yml](#). inspected revision.
4. Anthropic. [Effective context engineering for AI agents](#). 29 September 2025.
5. OpenAI, Ryan Lopopolo. [Harness engineering](#). 11 February 2026.
6. Lowerbeam. [README.md](#). inspected revision.
7. Lowerbeam. [chatStore.ts](#). inspected revision.
8. Lowerbeam. [chatClient.ts](#). inspected revision.
9. Lowerbeam. [ipc.ts](#). inspected revision.
10. Lowerbeam. [mcp.ts](#). inspected revision.
11. Lowerbeam. [index.ts: main process](#). inspected revision.
12. Electron. [Security: recommended practices](#). undated.
13. Pi project. [Agent core README](#). undated.
14. Pi project. [Coding agent SDK documentation](#). undated.
15. Pi project. [Custom model configuration](#). undated.
16. OpenCode. [Server documentation](#). undated.
17. OpenCode. [Providers: llama.cpp](#). undated.
18. OpenCode. [Permissions](#). undated.
19. OpenAI, Celia Chen. [Unlocking the Codex harness](#). 4 February 2026.
20. Electron. [utilityProcess API](#). undated.
21. Anthropic, Max McGuinness et al.. [How we contain Claude across products](#). 25 May 2026.
22. Anthropic. [sandbox-runtime repository and README](#). undated.
23. Model Context Protocol. [Security best practices](#). 2026-07-28 documentation.
24. Git. [git-worktree documentation](#). undated.
25. llama.cpp. [Function calling documentation](#). undated.
26. Anthropic, Prithvi Rajasekaran. [Harness design for long-running application development](#). 24 March 2026.
27. Agent Client Protocol. [Protocol overview](#). version 1 documentation.
28. Anthropic. [Demystifying evals for AI agents](#). 9 January 2026.